

DAT22-5

SQL & Data Querying

Albert School — 28 hours

Preface

In Pandas I (DAT12-3) you learned to take a table — a `DataFrame` — and interrogate it: filter rows with a boolean mask, summarise with `groupby`, stitch tables together with `merge`, and write a paragraph saying what the numbers mean. You already think in terms of **tables**, **conditions**, **groups**, and **joins**. That mental model is exactly the one this course rests on. What changes is the **medium**.

In Pandas the table lived in your computer's memory, inside a Python process, and you manipulated it with method calls. Here the table lives in a **database** — a program built for one job: storing related tables and answering questions about them, even when there are millions of rows and the data outlives any single script. You ask those questions in **SQL** (Structured Query Language), the language every database understands and the lingua franca of data work. A data scientist who cannot read and write SQL is cut off from where most of the world's structured data actually lives.

The good news is that almost every pandas idea has a SQL twin, and naming the correspondence up front will make the whole course feel like translation rather than starting over:

In Pandas I you wrote...	In SQL you write...
<code>df[df["amount"] > 100]</code>	<code>WHERE amount > 100</code>
<code>df.sort_values("amount")</code>	<code>ORDER BY amount</code>
<code>df.groupby("country").sum()</code>	<code>GROUP BY country + SUM(...)</code>
<code>a.merge(b, on="id")</code>	<code>a JOIN b ON a.id = b.id</code>
<code>df["c"].mean()</code>	<code>AVG(c)</code>

This course adds what a database gives you that an in-memory `DataFrame` cannot: **persistence** (the data stays put), **multiple related tables** enforced by the database itself, **transactional changes** (`INSERT`, `UPDATE`, `DELETE`), and **indexes** that keep queries fast as the data grows. By the end you will take a business question in plain English, turn it into a SQL query, run it against a real database from Python, and read the answer back in business terms — the foundation every machine-learning course after this one quietly assumes.

Contents

Preface	2
1 The relational model and your first <code>SELECT</code>	4
1.1 Why tables, and why more than one	4
1.2 The anatomy of a <code>SELECT</code>	5
1.3 <code>WHERE</code> : filtering rows	5
1.4 <code>ORDER BY</code> : sorting the result	6
1.5 <code>GROUP BY</code> and <code>HAVING</code> : from rows to groups	6
2 Aggregate functions	7
2.1 One number from many rows	7
2.2 Choosing the right aggregate	8
3 Joins	8
3.1 Putting the split-up tables back together	8
3.2 The four join types	9
3.3 Which join, and why it matters	9
3.4 Joining more than two tables	10

4	Subqueries and CTEs	10
4.1	When one query is not enough	10
4.2	Subqueries	10
4.3	Common Table Expressions (CTEs)	11
5	Creating and manipulating tables	12
5.1	Defining where the data lives	12
5.2	Inserting, updating, deleting	12
6	Database types: relational, warehouse, flat file	13
6.1	Different jobs, different stores	13
7	Connecting to SQLite from Python	14
7.1	SQL where you already work	14
8	Data quality at the query level	15
8.1	Clean the data where it lives	15
8.2	Duplicates	15
8.3	NULL handling	16
8.4	Inconsistent string formats	16
9	From business question to SQL	16
9.1	The translation skill	16
10	Indexing and query performance	17
10.1	When the table gets big	17
10.2	The trade-off: indexes are not free	18

1 The relational model and your first SELECT

1.1 Why tables, and why more than one

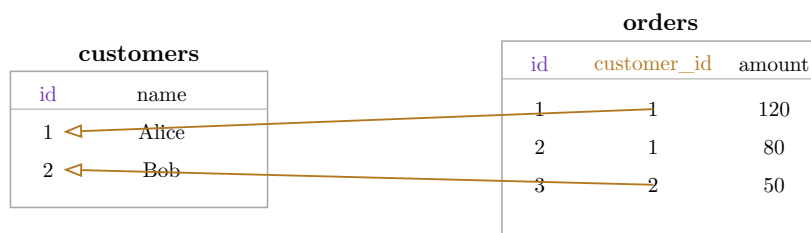
A **relational database** stores data in **tables** (also called **relations**): a grid of rows and columns, where each row is one record and each column one named, typed attribute. So far this is just a DataFrame. The leap is that a database holds **many** tables that **refer to each other**, and refuses to let those references rot.

Worked example — One table is not enough. Imagine recording online orders. You could put everything in one big table, repeating the customer’s name, email, and country on every single order:

order_id	customer_name	country	amount
1	Alice	France	120
2	Alice	France	80
3	Bob	Spain	50

Alice’s details are copied on every order she places. If she moves to Germany you must find and fix **every** row, and if you miss one the data now disagrees with itself. The relational answer is to split this into **two** tables — one for customers, one for orders — and let each order **point at** a customer.

Definition 1 (Primary key and foreign key) A **primary key** is a column (or set of columns) whose value **uniquely identifies** each row of a table — no two rows share it, and it is never NULL. A **foreign key** is a column in one table that **holds the primary key value of a row in another table**, creating a link between them. Foreign keys are how a relational database represents relationships without copying data.



foreign key → primary key

Figure 1: Two tables linked by a foreign key. `orders.customer_id` (orange) stores the value of `customers.id` (purple). Alice’s name lives in exactly one place; her two orders point at it. Change her name once and every order sees the update.

Remark This is the database version of **not repeating yourself**, and it is the single idea that distinguishes “data in a database” from “data in a spreadsheet”. Splitting data into linked tables to remove redundancy is called **normalisation**; you do not need its formal theory in this course, only the instinct: each fact should live in one place, and relationships are expressed by keys, not by copying.

1.2 The anatomy of a SELECT

A SELECT query **reads** rows from one or more tables. Nearly everything you do in this course is a SELECT. Its clauses always appear in this written order:

```
SELECT columns           -- which columns to return
FROM table               -- which table to read
WHERE condition          -- keep only rows matching this
GROUP BY columns         -- collapse rows into groups
HAVING group_condition   -- keep only groups matching this
ORDER BY columns         -- sort the result
LIMIT n;                -- return at most n rows
```

Worked example — Reading it like a sentence. “Show me the name and amount of each French sale over €100, biggest first”:

```
SELECT name, amount
FROM sales
WHERE country = 'France' AND amount > 100
ORDER BY amount DESC;
```

Read aloud it is almost English — which is the point of SQL. SELECT picks the columns, FROM the table, WHERE the rows, ORDER BY the order.

A crucial subtlety, and the source of many beginner errors: SQL is **written** in the order above, but the database **evaluates** the clauses in a **different** order. It starts from the table, filters rows, forms groups, filters groups, and only **then** works out which columns to show and how to sort them.



execution order — not the order you write the clauses in

Figure 2: The logical order in which a database evaluates a query. WHERE filters individual rows **before** groups exist; HAVING filters groups **after** they are formed; SELECT runs late, which is why a column alias defined in SELECT cannot usually be reused back in WHERE.

1.3 WHERE: filtering rows

WHERE keeps only the rows for which its condition is true. It runs **before** any grouping, so it sees individual rows.

```
SELECT name, country, amount
FROM sales
WHERE country = 'France' AND amount > 100;
```

Conditions combine with AND, OR, and NOT, and use the comparison operators you already know plus a few SQL-specific ones:

Operator	Meaning
= <> < > <= >=	equal, not-equal, and the orderings
BETWEEN a AND b	within a range, inclusive
IN (v1, v2, ...)	equals any value in the list
LIKE 'A%'	pattern match (% = any run of characters, _ = one)
IS NULL / IS NOT NULL	test for missing values

Common pitfall SQL strings are wrapped in **single** quotes: 'France'. Double quotes mean something different (an identifier, like a column name) in standard SQL. And equality uses a **single** =, not the == of Python. WHERE country == 'France' is a Python habit that many databases reject.

1.4 ORDER BY: sorting the result

ORDER BY sorts the rows the query returns. ASC (ascending) is the default; DESC reverses it. You can sort by several columns — ties on the first are broken by the second.

```
SELECT name, country, amount
FROM sales
ORDER BY country ASC, amount DESC;  -- by country A→Z, then biggest sale first
```

Pair ORDER BY with LIMIT to answer “top N” questions directly: sort, then take the first few rows.

```
SELECT name, amount
FROM sales
ORDER BY amount DESC
LIMIT 3;  -- the three largest sales
```

1.5 GROUP BY and HAVING: from rows to groups

GROUP BY is the heart of analytical SQL, and it is exactly the df.groupby you already know. It **collapses** all rows sharing the same value in the listed column(s) into a single group, so that an **aggregate function** (next chapter) can compute one summary number per group.

Worked example — Total revenue per country.

```
SELECT country, SUM(amount) AS revenue
FROM sales
GROUP BY country;
```

Every row with country = 'France' is folded into one France group; SUM(amount) then totals their amounts. The result has **one row per country**, not one row per sale.

HAVING then filters those **groups**, using a condition on the aggregate. This is the clause beginners most often miss, because it looks like it duplicates WHERE. The distinction is sharp and worth memorising:

Definition 2 (WHERE vs HAVING) WHERE filters rows, before grouping. It cannot mention an aggregate, because groups do not exist yet. HAVING filters groups, after grouping. It is the only place an aggregate condition such as SUM(amount) > 1000 may go.

Worked example — The two filters working together. “Among sales made in 2025, which countries had total revenue above €1000?”

```
SELECT country, SUM(amount) AS revenue
FROM sales
WHERE year = 2025  -- row filter: keep only 2025 sales
GROUP BY country
HAVING SUM(amount) > 1000  -- group filter: keep big-revenue countries
ORDER BY revenue DESC;
```

`WHERE` throws away non-2025 rows **before** totals are computed; `HAVING` inspects each country's total **after**. Swapping them is impossible: a `WHERE` on `SUM(amount)` is an error, and a `HAVING` on `year` would filter far too late.

Common pitfall Every non-aggregated column in the `SELECT` list of a grouped query must appear in `GROUP BY`. Writing `SELECT country, name, SUM(amount) ... GROUP BY country` is ambiguous — each country group contains many different names, so which one should the database show? Standard SQL rejects this; SQLite returns an arbitrary one, which is worse, because it looks like it worked. Group by every plain column you select, or aggregate it.

Chapter takeaways. A relational database stores many tables linked by primary/foreign keys, so each fact lives in one place. A `SELECT` is written `SELECT-FROM-WHERE-GROUP BY-HAVING-ORDER BY` but **evaluated** `FROM-WHERE-GROUP`

`BY-HAVING-SELECT-ORDER BY`. `WHERE` filters rows before grouping; `GROUP BY` collapses rows into groups; `HAVING` filters those groups on an aggregate; `ORDER BY` (+ `LIMIT`) sorts and trims.

2 Aggregate functions

2.1 One number from many rows

An **aggregate function** takes a whole column of values and returns a single summary value. On its own it summarises the entire table; with `GROUP BY` it summarises each group. These five are the workhorses, and the skill the OP asks for is not just using them but knowing **which one** a question calls for.

Definition 3 (The five core aggregates)

- `COUNT` — **how many** rows. `COUNT(*)` counts all rows; `COUNT(col)` counts rows where `col` is not `NULL`; `COUNT(DISTINCT col)` counts distinct non-`NULL` values.
- `SUM` — the **total** of a numeric column.
- `AVG` — the **mean** (average) of a numeric column.
- `MIN` — the **smallest** value.
- `MAX` — the **largest** value.

`SUM` and `AVG` require numbers; `COUNT`, `MIN`, and `MAX` work on any type (`MIN` of a text column is the alphabetically first value, `MIN` of a date the earliest).

Worked example — Profiling a column at a glance.

```
SELECT
  COUNT(*)      AS n_sales,
  SUM(amount)   AS total,
  AVG(amount)   AS mean,
  MIN(amount)   AS smallest,
  MAX(amount)   AS largest
FROM sales;
```

One row comes back, summarising the whole table: how many sales, their total, their typical size, and the extremes. This is the SQL equivalent of `pandas' df["amount"].describe()`.

2.2 Choosing the right aggregate

Each aggregate answers a different shape of business question. Matching the question to the function is the real skill:

The question asks for...	Use
how many customers / orders / distinct countries	COUNT (with DISTINCT for “distinct”)
total revenue, total units sold	SUM
typical order value, average rating	AVG
cheapest product, earliest signup	MIN
biggest order, most recent login	MAX

Common pitfall Three classic aggregate traps:

- **COUNT(*) vs COUNT(col)**. COUNT(*) counts rows; COUNT(col) ignores rows where col is NULL. To count **customers who have a phone number**, use COUNT(phone), not COUNT(*).
- **AVG silently skips NULLs**. AVG(amount) averages only the non-NULL amounts — it does **not** treat a missing value as zero. If a NULL should count as 0, write AVG(COALESCE(amount, 0)). The two can differ a lot.
- **Don’t SUM an average**. The average of per-day averages is **not** the overall average unless every day has the same number of rows. To get a true overall mean, AVG the raw rows, or compute SUM(...) / COUNT(...) yourself.

Chapter takeaways. Aggregates turn many rows into one summary value: COUNT (how many), SUM (total), AVG (mean), MIN/MAX (extremes). Pick the one that matches the question. Watch the NULL-handling differences — COUNT(*) counts rows while COUNT(col) and AVG(col) ignore NULLs — and never average pre-computed averages.

3 Joins

3.1 Putting the split-up tables back together

Chapter 1 split data across tables to avoid repetition. A **join** is how you bring the pieces back together for a query: it combines rows from two tables by **matching values** in a related column — almost always a foreign key in one table meeting the primary key in the other. This is exactly `df.merge` from Pandas I.

Worked example — Attaching each order to its customer. The orders table stores only a `customer_id`. To see the customer’s **name** beside each order, join the two tables on that key:

```
SELECT o.id, c.name, o.amount
FROM orders AS o
JOIN customers AS c ON o.customer_id = c.id;
```

AS o and AS c are **table aliases** — short nicknames so you can write `o.id` and `c.name`. The ON clause states the matching rule: pair an order with the customer whose `id` equals the order’s `customer_id`.

Remark Always qualify columns with the table alias (`o.amount`, `c.name`) in a join. If both tables have a column called `id` and you write a bare `id`, the database cannot tell which you mean and raises an “ambiguous column” error. Qualifying every column also makes the query far easier to read.

3.2 The four join types

The tables rarely match perfectly: some customers have no orders, and (in a messy database) some orders might reference a customer that no longer exists. What should happen to those **unmatched** rows? That single decision is what distinguishes the four join types.

Definition 4 (INNER, LEFT, RIGHT, FULL OUTER)

- **INNER JOIN** keeps only rows that have a match in **both** tables. Unmatched rows on either side are dropped.
- **LEFT JOIN** keeps **all** rows of the left table; where the right table has no match, its columns come back `NULL`.
- **RIGHT JOIN** keeps **all** rows of the right table; unmatched left columns are `NULL`.
- **FULL OUTER JOIN** keeps **all** rows from **both** tables; unmatched columns on whichever side are `NULL`.

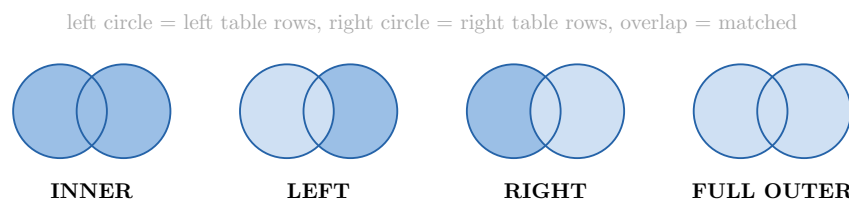


Figure 3: The four joins differ only in which unmatched rows they keep. **INNER** keeps the overlap (matched rows only); **LEFT** adds the unmatched left rows; **RIGHT** adds the unmatched right rows; **FULL OUTER** keeps everything. Shaded = returned by the join.

3.3 Which join, and why it matters

The choice is not cosmetic — it changes both the rows you get and the **numbers you compute from them**.

Worked example — Listing every customer, even the inactive ones. “List all customers and their total spend, including those who have never ordered.”

```
SELECT c.name, COALESCE(SUM(o.amount), 0) AS total_spend
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.id
GROUP BY c.name;
```

A **LEFT JOIN** from `customers` guarantees every customer appears. A customer with no orders gets `NULL` for the order columns; `SUM` of nothing is `NULL`, so `COALESCE(..., 0)` turns it into a clean `0`. Use an **INNER JOIN** here and the never-ordered customers vanish from the report entirely — a silent undercount.

Common pitfall The most dangerous join bug is **silent row loss**. An **INNER JOIN** quietly drops rows that don’t match — so if a few orders have a `customer_id` that isn’t in `customers`, those orders disappear and your `SUM(amount)` is too low, with no error to warn you. When

every row of one table must survive the join, reach for a `LEFT JOIN` from that table. A good habit: compare `COUNT(*)` before and after a join to see how many rows the match dropped.

Remark SQLite historically supported `INNER JOIN` and `LEFT JOIN` but **not** `RIGHT`

`JOIN` or `FULL OUTER JOIN` (recent versions, 3.39+, do). This matters little in practice: A `RIGHT JOIN B` is just `B LEFT JOIN A` with the tables swapped, and a `FULL OUTER JOIN` can be emulated by `UNION`-ing a `LEFT JOIN` with the unmatched rows of the other side. Master `LEFT JOIN` and you can express the rest.

3.4 Joining more than two tables

Real questions span several tables. You simply chain `JOIN` clauses; each adds one more table and one more `ON` condition.

```
SELECT c.name, p.name AS product, oi.quantity
FROM customers AS c
JOIN orders AS o      ON o.customer_id = c.id
JOIN order_items AS oi ON oi.order_id = o.id
JOIN products AS p    ON p.id = oi.product_id;
```

Chapter takeaways. A join recombines tables on a matching column (the foreign-key ↔ primary-key link). `INNER` keeps only matched rows; `LEFT` keeps all left rows (unmatched right columns become `NULL`); `RIGHT` mirrors it; `FULL OUTER` keeps everything. The choice changes which rows — and which totals — you get, so pick the join that preserves the rows your question requires. Always alias and qualify columns; chain joins for multi-table questions.

4 Subqueries and CTEs

4.1 When one query is not enough

Some questions need a query **whose answer feeds another query**. “Which sales are above average?” first needs the average, then the rows beating it. SQL gives two tools for this: the **subquery** (a query nested inside another) and the **CTE** (a named query you define up front). They can express the same things; CTEs are usually far more readable.

4.2 Subqueries

Definition 5 (Subquery) A **subquery** is a `SELECT` written inside another query. It can appear:

- in `WHERE`, to compare against a computed value or a set of values;
- in `FROM`, as a temporary table the outer query reads from;
- in the `SELECT` list, to compute a value per row.

Worked example — Sales above the overall average.

```
SELECT name, amount
FROM sales
WHERE amount > (SELECT AVG(amount) FROM sales);
```

The inner `SELECT AVG(amount) FROM sales` runs first and returns one number; the outer query then keeps every sale beating it. You could not do this with a plain `WHERE amount > AVG(amount)` — an aggregate is not allowed directly in `WHERE`, so the subquery is what lets you compare a row against a summary of the whole table.

A subquery that returns a **list** pairs with `IN`:

```
SELECT name
FROM customers
WHERE id IN (SELECT customer_id FROM orders WHERE amount > 500);
-- customers who have placed at least one order over €500
```

4.3 Common Table Expressions (CTEs)

As questions grow, nesting subqueries inside subqueries becomes unreadable — you have to read from the inside out. A **CTE** fixes this by letting you **name** a query and then refer to it by name, top to bottom, like defining a variable before using it.

Definition 6 (Common Table Expression (WITH)) A **CTE** is a named, temporary result set defined with a `WITH` clause at the start of a query and referenced by name in what follows. It exists only for the duration of that one query. Several CTEs are separated by commas, and each may build on the ones before it — letting you decompose a complex query into readable, named steps.

Worked example — The same question, as a readable pipeline. “Which countries have above-average total revenue?” — solved in two named steps:

```
WITH country_totals AS (
  SELECT country, SUM(amount) AS revenue
  FROM sales
  GROUP BY country
)
SELECT country, revenue
FROM country_totals
WHERE revenue > (SELECT AVG(revenue) FROM country_totals)
ORDER BY revenue DESC;
```

`country_totals` is computed once and named; the main query reads from it twice (once to list, once to average) as if it were a real table. Compare this with the same logic written as nested subqueries — the CTE version reads in the order you think.

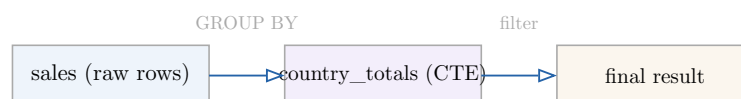


Figure 4: A CTE names an intermediate result so the query reads as a left-to-right pipeline — raw rows → grouped totals → filtered final answer — instead of an inside-out nest of subqueries.

Remark Reach for a subquery for a quick, one-off nested value (often in `WHERE`). Reach for a CTE when the query has several steps, when an intermediate result is used **more than once**, or simply when a name would make the logic clearer. Readable SQL is maintainable SQL, and CTEs are the main tool for keeping complex queries readable.

Chapter takeaways. A subquery is a `SELECT` nested inside another — in `WHERE` (compare against a value or list), `FROM`, or the select list — letting a row be compared against a summary of the data. A CTE (`WITH name AS (...)`) names an intermediate result so a complex query decomposes into readable, reusable, top-to-bottom steps. Prefer CTEs once a query has more than one step.

5 Creating and manipulating tables

5.1 Defining where the data lives

So far you have only **read** data. To build a database you must also **create** tables and **change** their contents. These four statements — `CREATE`, `INSERT`, `UPDATE`, `DELETE` — are the write side of SQL.

Worked example — Defining a table.

```
CREATE TABLE customers (  
  id      INTEGER PRIMARY KEY,  
  name    TEXT NOT NULL,  
  email   TEXT UNIQUE,  
  country TEXT  
);
```

Each line declares a column, its **type**, and optional **constraints**: `id` is the primary key (unique, not null, identifies the row); `name` may not be `NULL`; `email` must be unique across all rows; `country` is free to be anything, including missing.

Definition 7 (Column types and constraints) A **type** declares what kind of value a column holds — `INTEGER`, `REAL` (decimal), `TEXT`, and in SQLite the catch-all `BLOB`. A **constraint** is a rule the database **enforces** on every write:

- `PRIMARY KEY` — unique row identifier;
- `NOT NULL` — a value is required;
- `UNIQUE` — no two rows may share this value;
- `DEFAULT v` — value to use when none is given;
- `FOREIGN KEY (col) REFERENCES other(id)` — `col` must match a row in `other`.

Constraints are the database protecting your data's integrity **for** you — a guarantee an in-memory `DataFrame` cannot give.

5.2 Inserting, updating, deleting

```
-- add rows  
INSERT INTO customers (id, name, email, country)  
VALUES (1, 'Alice', 'alice@example.com', 'France'),  
       (2, 'Bob', 'bob@example.com', 'Spain');  
  
-- change existing rows  
UPDATE customers  
SET country = 'Germany'  
WHERE id = 1;  
  
-- remove rows
```

```
DELETE FROM customers
WHERE id = 2;
```

Common pitfall An **UPDATE** or **DELETE** without a **WHERE** clause hits every row in the table. `DELETE FROM customers;` empties the table; `UPDATE customers SET country = 'France';` makes everyone French. This is the single most destructive beginner mistake. Before running an **UPDATE/DELETE**, write the **WHERE** first, and ideally run the same **WHERE** inside a **SELECT** to see exactly which rows you are about to change before you change them.

Remark Statements that change data are wrapped in **transactions**. A transaction groups several changes so they either **all** succeed or **all** fail together, leaving the database consistent. When you work from Python (next chapter), changes are not permanent until you **commit** — until then you can **rollback** and undo them. This safety net is another thing a database gives you that editing a CSV by hand does not.

Chapter takeaways. `CREATE TABLE` defines columns with **types** and **constraints** (**PRIMARY KEY**, **NOT NULL**, **UNIQUE**, **FOREIGN KEY**) that the database enforces. `INSERT` adds rows, `UPDATE` changes them, `DELETE` removes them — and `UPDATE/DELETE` **without** a **WHERE** affect every row. Changes live inside transactions and become permanent only on **commit**.

6 Database types: relational, warehouse, flat file

6.1 Different jobs, different stores

A relational database is not the only place to keep data, nor always the right one. The OP asks you to tell three kinds apart and say when each fits. The deciding factor is the **workload**: many small reads and writes, or large read-mostly analytical scans, or simple portable storage.

Definition 8 (Three kinds of data store)

- A **relational database** (OLTP — online **transaction** processing) stores data in related tables and is tuned for many fast reads and writes of **individual rows**: an app recording orders as they happen. Examples: PostgreSQL, MySQL, SQLite.
- A **data warehouse** (OLAP — online **analytical** processing) stores large volumes of historical data tuned for queries that **scan and aggregate across millions of rows**: “total revenue per region per quarter for five years”. Examples: BigQuery, Snowflake, Redshift.
- A **flat file store** keeps data in plain files (CSV, JSON, Parquet) with **no query engine** of its own. Simple, portable, and ideal for small datasets, exchange between tools, or one-off storage.

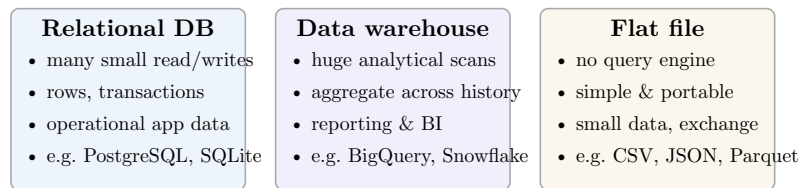


Figure 5: Choosing a store by workload: a relational database for live transactional row-level read/writes, a data warehouse for large-scale analytical aggregation over history, a flat file for small, portable, or one-off data.

Worked example — Matching store to scenario.

- A web shop recording each order the instant a customer checks out → **relational database** (frequent single-row writes, integrity constraints).
- The analytics team computing five years of sales trends every morning → **data warehouse** (massive aggregation, read-mostly).
- A 200-row lookup table you email to a colleague → **flat file** (CSV; no engine needed).

Remark The lines blur in practice — SQLite is a relational database that lives in a single file, and modern warehouses speak SQL just like relational databases do. The point is not rigid taxonomy but **matching the tool to the workload**: row-level transactions, large-scale analytics, or simple portable storage.

Chapter takeaways. A **relational database** suits many small transactional read/writes of individual rows; a **data warehouse** suits large analytical queries aggregating across vast history; a **flat file** suits small, portable, engine-free storage. Choose by the workload, not by habit.

7 Connecting to SQLite from Python

7.1 SQL where you already work

A query is only useful if you can run it from your analysis code and feed the result into pandas, a model, or a report. **SQLite** is a complete relational database that lives in a single file and needs no server, and Python’s built-in `sqlite3` module talks to it directly — no installation required. This is the bridge between this course and every Python-based course around it.

Definition 9 (Connection and cursor) A **connection** (`sqlite3.connect("file.db")`) is your session with the database. A **cursor** (`conn.cursor()`) is the object you use to **execute** statements and **fetch** their results: `execute(sql)` runs a statement; `fetchone()` returns the next result row, `fetchall()` returns all remaining rows as a list of tuples.

Worked example — A full round-trip.

```
import sqlite3

conn = sqlite3.connect("shop.db")    # open (creates the file if absent)
cursor = conn.cursor()

cursor.execute(
```

```

    "SELECT name, amount FROM sales WHERE country = ?",
    ("France",), # parameters fill the ? placeholders
)
for name, amount in cursor.fetchall(): # iterate the result rows
    print(name, amount)

conn.commit() # persist any INSERT/UPDATE/DELETE made this session
conn.close() # release the file

```

execute sends the query; fetchall pulls the rows back into Python. commit makes write-changes permanent; close ends the session.

Common pitfall Never build SQL by string-formatting user input. Writing `f"WHERE name = '{name}'"` is the classic **SQL injection** hole: a name of `'; DROP TABLE sales;` -- rewrites your query into a destructive one. Always pass values as parameters with `?` placeholders and a tuple — `execute("... WHERE name = ?", (name,))` — so the database treats them strictly as data, never as code. (Note the one-element tuple `(name,)` **needs** the comma.)

Remark In data work the most common shortcut is to skip the cursor entirely and let pandas do the round-trip: `pd.read_sql_query("SELECT ...", conn)` returns the result straight as a DataFrame, and `df.to_sql("table", conn)` writes one back. Under the hood it is the same `sqlite3` connection — handy once you are comfortable with the explicit cursor shown above.

Chapter takeaways. `sqlite3.connect` opens a single-file database; a **cursor** runs statements (`execute`) and retrieves rows (`fetchone`/`fetchall`). Persist writes with `commit`, finish with `close`. Always pass values via `?` parameters, never string-formatting — that prevents SQL injection. For analysis, `pd.read_sql_query` returns results directly as a DataFrame.

8 Data quality at the query level

8.1 Clean the data where it lives

Pandas I taught you to clean a DataFrame after loading it. Often it is better to catch problems **in the query itself**, at the source — three issues recur: duplicates, NULLs, and inconsistent strings. Each has a SQL remedy.

8.2 Duplicates

Duplicate rows inflate counts and totals. Find them by grouping on the columns that **should** be unique and counting:

```

SELECT email, COUNT(*) AS n
FROM customers
GROUP BY email
HAVING COUNT(*) > 1;    -- emails that appear more than once

```

To return a deduplicated result, `SELECT DISTINCT` drops exact-duplicate rows:

```

SELECT DISTINCT name, email, country FROM customers;

```

8.3 NULL handling

Definition 10 (What NULL means) NULL is not zero and not an empty string — it means **unknown** / **missing**. Any arithmetic or comparison with NULL yields NULL (effectively “unknown”), so NULL = NULL is **not** true. You therefore test for it with IS NULL and IS NOT NULL, never = NULL, and substitute a value with COALESCE(col, replacement), which returns the first non-NULL argument.

Worked example — Testing and replacing NULLs.

```
SELECT name, COALESCE(country, 'Unknown') AS country
FROM customers
WHERE country IS NULL;           -- the rows missing a country, labelled 'Unknown'
```

Common pitfall The deepest NULL trap is that WHERE country <> 'France' **excludes** the rows where country is NULL — because NULL <> 'France' is NULL, not true. If you want non-French customers **including** those with no country recorded, you must say so explicitly: WHERE country <> 'France' OR country IS NULL. Forgetting this silently drops rows from your result.

8.4 Inconsistent string formats

The same value typed three ways — "France", "france ", " FRANCE" — counts as three distinct groups, scattering your aggregates. Normalise with string functions:

```
SELECT DISTINCT LOWER(TRIM(country)) AS country
FROM customers;
-- TRIM removes surrounding spaces; LOWER standardises case
```

TRIM strips leading/trailing whitespace, LOWER/UPPER unify case, and || concatenates. Applying these inside GROUP BY makes the messy variants collapse into one clean group.

Chapter takeaways. Fix data-quality issues at query level: find **duplicates** with GROUP BY ... HAVING COUNT(*) > 1 and drop them with DISTINCT; treat **NULL** as unknown — test with IS NULL, substitute with COALESCE, and remember inequality filters silently exclude NULLs; normalise **inconsistent strings** with TRIM/LOWER before grouping.

9 From business question to SQL

9.1 The translation skill

The whole course converges here: turning a question asked in English into a query, running it, and reporting the answer in business terms. This is a repeatable procedure, not a flash of insight.

Definition 11 (Question → query in five steps)

1. **Identify the tables** that hold the relevant facts (and how they join).
2. **Filter the rows** the question restricts to → WHERE.
3. **Decide the grouping** — is the answer one number, or one per category? → GROUP BY.
4. **Choose the summary** — count, total, average, extreme? → an aggregate.

5. **Order and limit** for “top/bottom N”, then **interpret** the result in the words of the original question.

Worked example — "Which three countries generated the most revenue in 2025?".

Walk the steps: the facts are in **sales**; restrict to 2025 (**WHERE**); the answer is one number **per country** (**GROUP BY** country); the summary is total revenue (**SUM**); we want the top three (**ORDER BY ... DESC LIMIT 3**).

```
SELECT country, SUM(amount) AS revenue
FROM sales
WHERE year = 2025
GROUP BY country
ORDER BY revenue DESC
LIMIT 3;
```

Interpreting the result: the three rows returned are the firm’s largest markets by revenue in 2025 — where, in plain terms, the money came from that year. A query is only finished once you can say what its rows **mean** to the business, not merely that they appeared.

Common pitfall A query that runs is not the same as a query that is **correct**. The dangerous failures are silent: an **INNER JOIN** that dropped unmatched rows, a **NULL** that skewed an **AVG**, a missing **WHERE** that counted every year instead of one. Sanity-check every answer — does the row count look plausible? Does the total roughly match a back-of-envelope estimate? An answer you cannot defend in business terms is not an answer yet.

Chapter takeaways. Translate a business question by naming the tables, filtering rows (**WHERE**), choosing the grouping (**GROUP BY**), picking the aggregate, then ordering/limiting — and finish by interpreting the rows in business language. Always sanity-check: a query that runs can still be quietly wrong.

10 Indexing and query performance

10.1 When the table gets big

On a thousand rows every query is instant. On ten million, a query that filters on an unindexed column must read **every** row to find the matches — a **full table scan**. An **index** is the database’s answer, and the trade-offs it carries are the last idea the course needs.

Definition 12 (Index) An **index** is an auxiliary data structure (typically a balanced tree) that keeps the values of one or more columns in sorted order, with pointers to the rows that contain them. It lets the database **jump straight** to the matching rows instead of scanning the whole table — turning a search from $O(n)$ (read every row) into roughly $O(\log n)$ (descend a tree).

```
CREATE INDEX idx_sales_country ON sales(country);
```

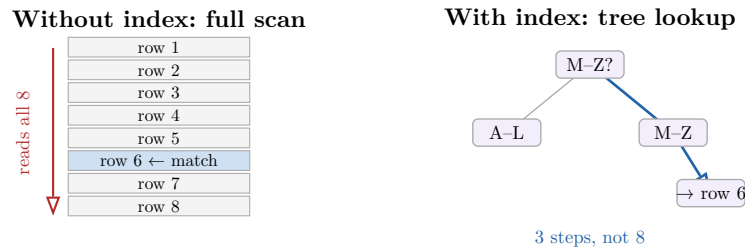


Figure 6: Searching for one value. Without an index the database reads every row (a full scan, $O(n)$). With an index it descends a sorted tree, touching only a few nodes ($O(\log n)$) — a gap that widens enormously as the table grows.

10.2 The trade-off: indexes are not free

An index would seem to make everything faster, so why not index every column? Because an index is a **second copy** of the data that the database must **keep in sync**.

Definition 13 (What an index costs) An index speeds up **reads** that filter, join, or sort on the indexed column — but it **slows down writes** (INSERT, UPDATE, DELETE), because every write must also update the index, and it **consumes extra storage**. An index is a bet that the column will be searched far more often than it is written.

Worked example — When to create one, when not to. Worth indexing: the `customer_id` column of a million-row `orders` table that you constantly filter and join on — the read speed-up is huge and well worth the write cost.

Not worth it: a `is_active` boolean with only two values (an index barely narrows the search), a tiny 50-row lookup table (a full scan is already instant), or a column you never filter on (you pay the write and storage cost for no read benefit).

Common pitfall Do not reflexively index everything. On a write-heavy table, redundant indexes can make the system **slower overall** — every insert now maintains a stack of indexes nobody queries. Index the columns that appear in your **WHERE** and **JOIN** conditions on **large** tables, measure the effect (most databases offer an **EXPLAIN** command to confirm an index is actually being used), and add no more than the queries justify.

Chapter takeaways. An index is a sorted auxiliary structure that turns a filter/join lookup from a full scan ($O(n)$) into a fast tree descent ($O(\log n)$). It speeds reads but slows writes and costs storage, so index the columns frequently filtered or joined on **large** tables — not tiny tables, low-variety columns, or columns you never search. Measure with **EXPLAIN**.

Exercises

1. On a real database, write one **SELECT** that uses **WHERE**, **GROUP BY**, **HAVING**, and **ORDER BY** together. State, for each clause, whether it acts on rows or on groups and why it must run where it does.
2. Using a single query, compute **COUNT(*)**, **SUM**, **AVG**, **MIN**, and **MAX** of a numeric column. Then write one business question that each of the five aggregates answers, and explain why **COUNT(*)** and **COUNT(col)** can differ.
3. Take a **customers** and an **orders** table. Run the same query with an **INNER JOIN** and then a **LEFT JOIN**, and explain exactly which rows differ and why. Then compute total spend per customer including customers with no orders, using **COALESCE** to turn the missing total into 0.
4. Write a query answering “which sales are above the overall average?” first with a subquery, then rewrite it using a CTE. Explain when you would prefer each form.
5. **CREATE** a table with a primary key, a **NOT NULL** column, and a **UNIQUE** column. **INSERT** several rows, **UPDATE** one, and **DELETE** another. Then describe what would happen if you ran your **UPDATE** or **DELETE** without its **WHERE** clause.
6. For three scenarios of your choosing, state whether a relational database, a data warehouse, or a flat file store is the right choice, and justify each by the workload it implies.
7. From Python, connect to a SQLite database with **sqlite3**, run a **parameterised** query with a **?** placeholder, and print the rows. Explain why the parameterised form is safer than building the SQL string with an f-string.
8. On a deliberately messy table, write three queries: one that finds duplicate rows, one that replaces **NULLs** with **COALESCE**, and one that normalises inconsistent string casing/whitespace before grouping. Show how a **<>** filter silently drops **NULL** rows and fix it.
9. Translate a business question of your own into SQL using the five-step procedure, execute it, and write two sentences interpreting the result in business terms — including one sanity check you ran on the answer.
10. Create an index on a column, and explain in your own words a query it would speed up and a situation (a small table, a low-variety column, or a write-heavy table) where adding it would be a mistake. If your tool supports **EXPLAIN**, confirm the index is used.